

GOMMAGE

AGENT EXECUTION TRACER & DETERMINISTIC REPLAY ENGINE

TECHNICAL ARCHITECTURE & DESIGN JUSTIFICATION

THE VISION

Imagine this — entirely inside Jira: An agent fails on a sensitive ticket. Its entire trajectory — prompts, tool calls, memory — is automatically recorded and attached to the ticket. A developer opens the ticket, clicks "Replay in Debug Mode". The tool call that would have sent an email is automatically mocked. The developer rewrites the prompt at step 4, hits Continue, and watches the agent take a safer path — still using the original recorded responses. Once satisfied, they create a linked Jira issue for the fix, with the debug trace as evidence.

TEAM

Clair Obscur

AUTHORS

Ala Eddine Zaouali

(alaeddine.zaouali@insat.ucar.tn)

Adem Saidi (adem.saidi@insat.ucar.tn)

TRACK

AINS Hackathon 2026, Use Case 2

DATE

June 2026

VERSION

1.0 — First Submission

Abstract

AI agents deployed in enterprise Atlassian environments fail in ways that are fundamentally non-reproducible using classical debugging methods. When an agent re-executes, the LLM produces different outputs; worse, re-execution may trigger side-effecting tools — sending emails, modifying tickets — during the debug session itself. **gommage** is an infrastructure-layer proxy that (i) transparently records the full trajectory of any LLM agent run as a structured *Agent Execution Record* (AER), (ii) replays that trajectory deterministically by intercepting and mocking all tool calls, (iii) surfaces this workflow natively inside Jira, and (iv) employs a transport-layer verification mechanism intercepting calls at the stdio, shell, and filesystem layers to reconcile self-reported AER fields. This document justifies the technical choices underpinning gommage, grounds them in the AER provenance model of Vispute (2026), and presents the 5-day implementation plan.

CONTENTS

1	Problem Framing	2
1.1	The Debugging Gap	
1.2	Scope and Target System	
2	Theoretical Foundation	2
2.1	The Agent Execution Record (AER)	
3	Technical Architecture	3
3.1	System Architecture & Flow	
3.2	Pipeline Mechanics	
3.3	AER Schema Definition	
4	Technology Choices & Justification	4
5	Feature Design & Divergence Tracking	4
5.1	Mock Registry & Safety Guards	
5.2	Divergence Editing	
6	Evaluation Methodology	5
7	5-Day Implementation Plan	5
8	Conclusion	5

1. PROBLEM FRAMING

1.1. THE DEBUGGING GAP

Classical software debugging relies on two guarantees: *reproducibility* (running the same code on the same input produces the same state) and *isolation* (reproducing a bug does not cause additional side effects in production). Both guarantees collapse for LLM-based agents.

- **Non-reproducibility:** The same instruction passed to an LLM agent twice yields different tool call sequences, reasoning chains, and outputs due to sampling stochasticity. Re-running a failed agent to debug it produces a different failure — or no failure at all.
- **Side effects during debug:** A failed Jira triaging agent, re-run by an engineer, may send a customer email or overwrite a ticket during the debug session — compounding the original damage.
- **Opaque state:** Simple log files capture what happened but not *why*: what prompt the LLM received at each step, what context was in its window, and what it inferred before each action.

What is missing is an enterprise-grade **Flight Recorder**: infrastructure that captures every LLM input, context variable, and tool-call payload during a live run, then allows developers to re-execute the agent step-by-step using the exact recorded tool responses — guaranteeing determinism without touching live APIs.

1.2. SCOPE AND TARGET SYSTEM

gommage targets agents deployed in the Atlassian ecosystem (Jira Software, Jira Service Management) that interact with external tools (databases, email, webhooks). The demo agent is a **Jira ticket triaging agent** that reads incoming tickets, queries a database for SLA context, and routes tickets to teams — with an email notification on completion.

2. THEORETICAL FOUNDATION

2.1. THE AGENT EXECUTION RECORD (AER)

Our core data model is grounded in Vispute (2026), *Reasoning Provenance for Autonomous AI Agents* [1]. Vispute formalises a distinction that is central to our design:

*"State checkpoint systems save serialized snapshots of the agent's computational state. What they do not contain as a structured field is **why** the agent chose each action, **what it concluded** from each observation, or **how each conclusion shaped its strategy**."*

The AER model introduces structured reasoning provenance as a first-class schema primitive. Vispute further defines three replay modes — **mock replay** being the one gommage implements — which re-executes the agent using recorded tool responses without calling live endpoints.

We extend the AER concept in two directions suited to our Jira-native context:

1. **Side-effect classification field:** Each tool call record is tagged with a `side_effect` boolean and a `mock_required` flag, enabling the Mock Registry to automatically intercept

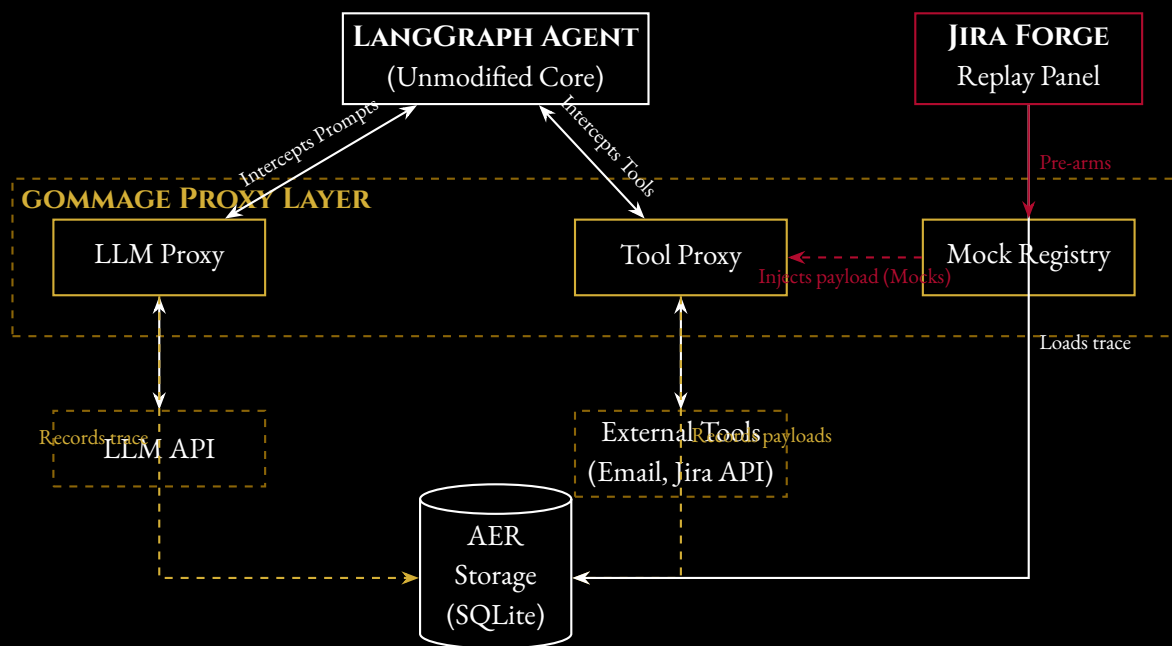
destructive calls during replay.

2. **Jira attachment envelope:** The AER is serialised with a top-level `jira_ticket_id` field and pushed as a structured attachment to the originating ticket via the Jira REST API.

3. TECHNICAL ARCHITECTURE

3.1. SYSTEM ARCHITECTURE & FLOW

The system splits into a **Recording Pipeline** (during live operations) and a **Replay Pipeline** (during debugging). Figure 1 visualizes how the Proxy layers sit between the Agent, the LLM, and external Tools without requiring code modifications to the Agent itself.



gommage Architectural Flow. The proxy layer seamlessly intercepts LLM and tool calls. During replay, the Jira UI arms the Mock Registry to inject recorded payloads instead of hitting external tools.

3.2. PIPELINE MECHANICS

During a **live run**, the unique `run_id` binds to the Jira ticket. Every call to the Anthropic API is captured as an `LLMStep`, and every tool execution is captured as a `ToolStep`. Upon completion, the AER Serialiser assembles these into a JSON structured trace attached natively to Jira.

During a **replay run**, the developer triggers execution from the Jira Replay Panel. The Mock Registry pre-loads all tools tagged `side_effect: true`. As the agent walks the graph, the Tool Proxy intercepts external calls, queries the Mock Registry, and safely returns the exact output the agent saw originally.

3.3. AER SCHEMA DEFINITION

To guarantee Pydantic v2 validation and robust typing, gomme utilizes the following structure:

```
class AERStep(BaseModel):
    step_id: int
    step_type: Literal["llm_call", "tool_call"]

    # LLM Context
    messages: Optional[List[dict]]
    response: Optional[str]

    # Tool Context
    tool_name: Optional[str]
    tool_input: Optional[dict]
    tool_output: Optional[Any]
    side_effect: bool = False           # Flag for destructive calls
    mock_required: bool = False        # Enforces mock on replay

    # Reasoning Provenance (Vispute 2026)
    intent: Optional[str]
    inference: Optional[str]
    evidence_chain: Optional[list]

class AgentExecutionRecord(BaseModel):
    run_id: str
    jira_ticket_id: str
    steps: List[AERStep]
    status: Literal["success", "failure", "partial"]
```

4. TECHNOLOGY CHOICES & JUSTIFICATION

- **LangGraph & LLM API:** LangGraph's checkpointing provides natural hooking points for graph super-steps, whilst LLM Sonnet excels at structured output generation and maintains high fidelity across long context arrays during deterministic replay.
- **Atlassian Forge (UI Kit 2):** The Jira-native replay panel runs serverlessly inside Atlassian's infrastructure. Forge's `@forge/api` allows direct access to the AER attachment, ensuring sensitive trace data never leaves the customer's tenant boundary.
- **SQLite (Demo) to PostgreSQL (Prod):** Local development leverages SQLite for immediate portability. Production scale-out utilizes PostgreSQL to facilitate cross-run behavioral analytics [1].

5. FEATURE DESIGN & DIVERGENCE TRACKING

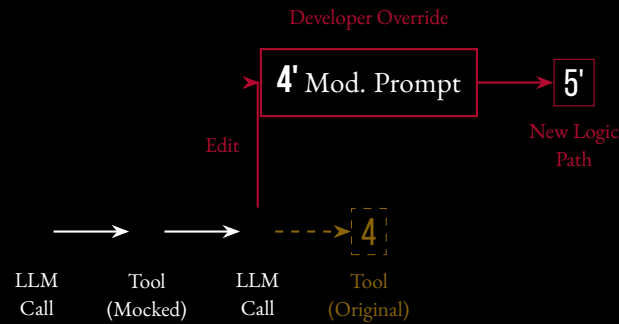
5.1. MOCK REGISTRY & SAFETY GUARDS

The Mock Registry prevents live tool execution during replay. Detection uses both static patterns (`send_email`, `execute_sql_write`) and dynamic AER-driven classification (HTTP 201/202 status codes observed during the live run). A **MockBadge** UI component renders inside Jira

so developers explicitly see which API calls were suppressed.

5.2. DIVERGENCE EDITING

The true power of gomme lies in Divergence Editing. A developer can modify the prompt at step N . The LLM produces a new output, branching the trajectory. From $N + 1$ onward, gomme continues to supply recorded tool responses where applicable, but the agent’s logic pathway updates.



Divergence Tracking. The developer edits the prompt at Step 4, causing the Replay Engine to fork the state tree. Original mocked data (Steps 1–3) remains intact, isolating the specific logic failure.

6. EVALUATION METHODOLOGY

We define two primary metrics to assess gomme’s correctness during the hackathon testing phase.

REPLAY FIDELITY SCORE (RFS) — measures whether replay re-presents identical contexts.

$$RFS \geq 0.95$$

Target: $RFS \geq 0.95$ on synthetic test suite.

MOCK RECALL RATE (MRR) — measures whether all side-effecting tool calls are intercepted.

$$MRR = 1.0$$

Target: $MRR = 1.0$ (Zero false negatives — absolutely no live calls during replay).

7. 5-DAY IMPLEMENTATION PLAN

8. CONCLUSION

gomme addresses a structural gap in enterprise AI agent infrastructure: the inability to reproduce, inspect, and safely debug LLM agent failures in production. By grounding the system in the AER provenance model [1], building a transparent proxy that requires no modifications to the agent’s core architecture, and surfacing the entire debugging workflow natively inside Jira, we deliver a system where AI is the mechanism — not a feature. Remove the structured recording, deterministic replay, and side-effect mock layer, and the debugging problem is unsolved. That is the bar the specification sets, and that is the bar gomme is built to clear.

EXECUTION PLAN (LLM CODE ACCELERATED)

DAY	FOCUS	KEY DELIVERABLES
1	Foundation	AER Pydantic schema; Proxy stubs; SQLite setup; Demo agent skeleton.
2	Recording Pipe	Full trajectory recording; side-effect auto-classification; Jira attachment via REST.
3	Replay Engine	Replay Runner; Pre-armed Mock Registry; Divergence Tracker core logic.
4	Jira UI	Forge panel scaffold; StepViewer; PromptEditor; DiffViewer component.
5	Eval & Polish	Run Scenarios A, B, C; Calculate RFS & MRR; Finalize README & Demo Video.

REFERENCES

- [1] Vispute, N. (2026). *Reasoning Provenance for Autonomous AI Agents: Structured Behavioral Analytics Beyond State Checkpoints and Execution Traces*. arXiv:2603.21692 [cs.AI]. Oracle Cloud Infrastructure.
- [2] AINS 4.0 & Vectors. (2026). *AINS Hackathon 2026 — Technical Specification*. Use Case 2: Agent Execution Tracer and Deterministic Replay Engine.
- [3] LangChain, Inc. (2024). *LangGraph: Build Stateful, Multi-Actor Applications with LLMs*. <https://langchain-ai.github.io/langgraph/>
- [4] Atlassian. (2024). *Forge Developer Documentation — Jira UI Kit 2*. <https://developer.atlassian.com/platform/forge/>